Java on the Command Line

The Concepts

Haluk O. Bingol

Faculty of Computer and Information Sciences
Yeditepe University

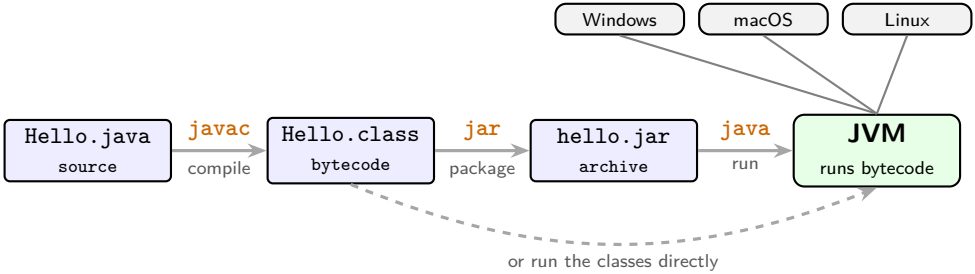
v2026-10-13

Outline

1. The Big Picture
2. Getting a JDK
3. How the Shell Finds the Tools
4. Compiling and Running
5. Packaging
6. Summary

The Big Picture

From source code to a running program



- **Compile once**: `javac` turns Java source into **bytecode**, instructions for a virtual machine, not for a real CPU
- **Run anywhere**: every operating system has its own JVM that executes the same `.class` and `.jar` files
- The command line is where all of this happens; IDEs (Eclipse, IntelliJ, VS Code) run the same tools for you

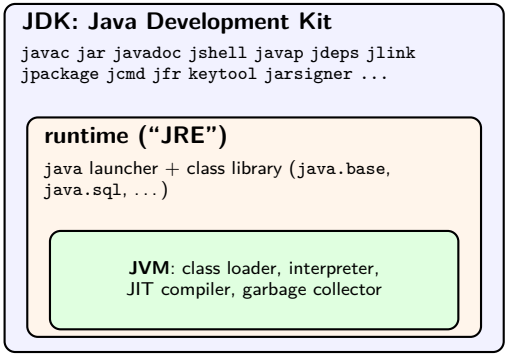
The whole pipeline in six commands

Terminal codeoutput/Pipeline.txt

```
$ javac -d classes src/edu/yeditepe/hello/Hello.java
$ find classes -type f
classes/edu/yeditepe/hello/Hello.class
$ jar --create --file hello.jar --main-class edu.yeditepe.hello.Hello -C classes .
$ jar --list --file hello.jar
META-INF/
META-INF/MANIFEST.MF
edu/
edu/yeditepe/
edu/yeditepe/hello/
edu/yeditepe/hello/Hello.class
$ java -jar hello.jar Ayşe
Hello, Ayşe!
$ java -cp classes edu.yeditepe.hello.Hello Can
Hello, Can!
```

A real session ($ marks what was typed). The package `edu.yeditepe.hello` became the folders `edu/yeditepe/hello/`; the JAR is a ZIP file plus a `MANIFEST.MF` that names the main class.

JDK, JRE and JVM



- **JVM**: the virtual machine that executes bytecode
- **Runtime** (JRE): JVM + standard library + the `java` command; enough to *run*
- **JDK**: runtime + development tools; needed to *compile and package*

Install a JDK

Students need the JDK. Since Java 11 Oracle no longer ships a separate JRE; some vendors still offer JRE packages, and `jlink` builds a minimal runtime for one application.

What is inside a JDK

Terminal codeoutput/JdkTools.txt

```
$ java -version
openjdk version "21.0.12.1" 2026-08-18
OpenJDK Runtime Environment (build 21.0.12.1+1-1-24.04.4-Ubuntu)
OpenJDK 64-Bit Server VM (build 21.0.12.1+1-1-24.04.4-Ubuntu, mixed mode, sharing)
$ javac -version
javac 21.0.12.1
$ ls "$JAVA_HOME/bin" | xargs -n 8
jar jarsigner java javac javadoc javap jcmd jconsole
jdb jdeprscan jdeps jfr jhsdb jimage jinfo jlink
jmap jmod jpackage jps jrunscript jshell jstack jstat
jstatd jwebserver keytool rmiregistry serialver
```

This machine runs Linux with an OpenJDK 21 build from Ubuntu; on Windows and macOS the same tools are in the JDK's `bin` folder (on Windows with `.exe`: `javac.exe`, `java.exe`, ...).

The main tools

Tool	Does	Typical use
`javac`	compiles `.java` to `.class`	`javac -d classes Hello.java`
`java`	starts a JVM and runs a program	`java -cp classes Hello`, `java -jar app.jar`
`jar`	creates and lists JAR archives	`jar -create -file app.jar -C classes .`
`javadoc`	HTML documentation from comments	`javadoc -d docs src/*.java`
`jshell`	interactive Java (read–eval–print loop)	trying out expressions
`javap`	disassembles `.class` files	`javap -c Hello`
`jdeps`	which packages and modules a JAR needs	`jdeps app.jar`
`jlink`	builds a custom, smaller runtime	`jlink -add-modules java.base ...`
`jpackage`	native installer (`.exe`, `.msi`, `.dmg`, `.pkg`, ...)	shipping to end users
`jcmd`, `jfr`	inspects running JVMs	diagnostics, profiling

Getting a JDK

One source code, many distributions

OpenJDK is the open-source project that develops Java. Several vendors build, test and ship it:

Build	From
Eclipse Temurin	Adoptium, `adoptium.net`
Oracle JDK	Oracle
Oracle OpenJDK builds	`jdk.java.net`
Microsoft Build of OpenJDK	Microsoft
Amazon Corretto	Amazon
Azul Zulu	Azul
BellSoft Liberica	BellSoft

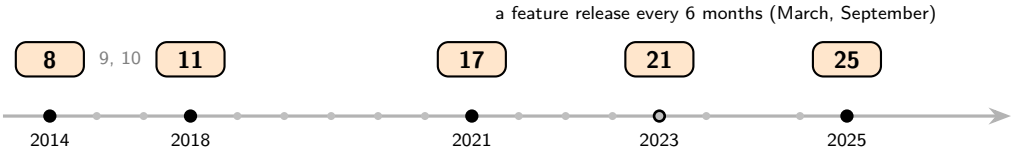
They differ in

- how long a version gets security updates
- license terms (read them for commercial use)
- platforms and packaging (installer, ZIP, package manager)
- extras, e.g. some bundle JavaFX

For this course

Any current JDK works. These slides use **Eclipse Temurin**: free, with installers for Windows and macOS.

Versions and long-term support



- Since Java 10 (2018), a new version every **six months**; most are supported only until the next one
- Every two years (since 17: 17, 21, 25) a **long-term support** (LTS) release, shown boxed, gets updates for years
- Use an **LTS** for courses and production: Java 21 or 25 today. Examples here were run with Java 21
- `java -version` tells you what you are running; `21.0.12` means version 21, update 12

Ways to install a JDK

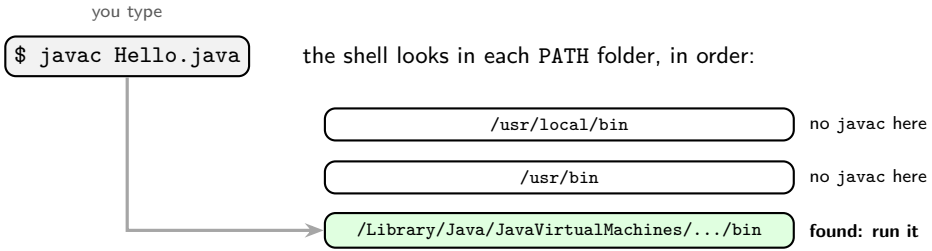
	Windows	**macOS**
Installer	`.msi` from `adoptium.net`; tick *Set JAVA_HOME* and *Add to PATH*	`.pkg` from `adoptium.net` (Apple silicon: *aarch64*; Intel: *x64*)
Package manager	`winget install EclipseAdoptium.Temurin.21.JDK`	`brew install --cask temurin@21`
Archive	unzip the `.zip`; set `PATH` yourself	unpack the `.tar.gz`; set `PATH` yourself
Installed in	`C:\Program Files\Eclipse Adoptium\jdk-21...`	`/Library/Java/JavaVirtualMachines/`
Check	`java -version`, `where java`	`java -version`, `/usr/libexec/java_home -V`

Close and reopen the terminal after installing

A terminal reads `PATH` when it starts; one that was open during the installation still cannot find `javac`.

How the Shell Finds the Tools

How the shell finds `javac`: `PATH`



- `PATH` is an **environment variable**: a list of folders, searched from first to last
- The first match wins, so with two JDKs installed, **order matters**
- Not found anywhere: **Windows** `'javac' is not recognized ...`, **macOS** `zsh: command not found: javac`
- `JAVA_HOME` is another variable: the JDK folder itself. Build tools (Maven, Gradle) and IDEs use it

The same ideas, different syntax

	Windows (Command Prompt)	**macOS (Terminal, zsh)**
show a variable	`echo %JAVA_HOME%`	`echo $JAVA_HOME`
where is a command	`where javac`	`which javac`
folder separator	`\`: `src\Hello.java`	`/`: `src/Hello.java`
path list separator	`;` (`-cp classes;lib\a.jar`)	`:` (`-cp classes:lib/a.jar`)
list files	`dir`	`ls`
current folder	`cd`	`pwd`
last exit code	`echo %ERRORLEVEL%`	`echo $?`
set for this session	`set NAME=value`	`export NAME=value`
set permanently	*System Properties, Environment Variables*	a line in `~/.zshrc`

`java`, `javac` and `jar` accept `/` in file paths on Windows too; the **path list** separator (`;` versus `:`) is the difference that really matters.

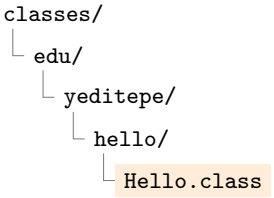
Compiling and Running

Packages are folders

```
package edu.yeditepe.hello;

public class Hello { ... }
```

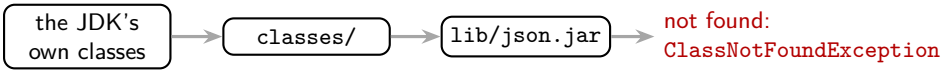
- The **fully qualified name** is `edu.yeditepe.hello.Hello`
- `javac -d classes` writes the class to `classes/edu/yeditepe/hello/Hello.class`
- `java` takes the **class name**, not a file name: `java -cp classes edu.yeditepe.hello.Hello`



the class-path entry is `classes`, the rest is the package

The class path: where `java` looks for classes

```
$ java -cp classes:lib/json.jar edu.yeditepe.app.Main
```



searched in this order; the first match wins

- The **class path** lists folders and JAR files: `-cp` (or `-classpath`, `-class-path`)
- Default: the current folder `.` (unless the `CLASSPATH` variable is set; better leave it unset)
- **Windows** separates entries with `;`, **macOS** (and Linux) with `:`
- `javac` uses a class path too, to find the classes your code uses

Running a source file directly (Java 11+)

Classic: compile, then run

```
javac Hello.java      # -> Hello.class
java Hello            # run the class
```

Source-file mode

```
java Hello.java       # compile + run
```

- `java Hello.java` compiles the file **in memory**; no `.class` file is written
- Java 11 to 21: the program must be a **single file**
- Java 22+: it may use other source files in the same folder tree
- Great for small programs and scripts; real projects still compile with `javac` and ship JARs

Packaging

A JAR is a ZIP file with a manifest

hello.jar

```
META-INF/MANIFEST.MF
edu/yeditepe/hello/Hello.class
edu/yeditepe/hello/Util.class
messages.properties
images/logo.png
```

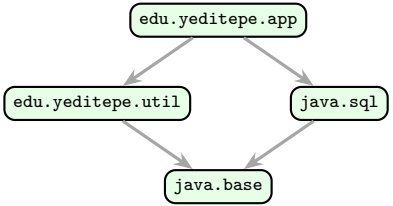
`META-INF/MANIFEST.MF`:

```
Manifest-Version: 1.0
Main-Class: edu.yeditepe.hello.Hello
Class-Path: lib/json.jar
```

- One file instead of a folder tree: classes, resources (text, images), metadata
- **Compressed** like a ZIP; any ZIP tool can open it
- `Main-Class` makes it **executable**: `java -jar hello.jar`
- `Class-Path` names other JARs it needs, relative to the JAR's folder
- A **library** JAR has no `Main-Class`; programs put it on their class path
- A "fat" (uber) JAR copies all libraries inside: one file to ship

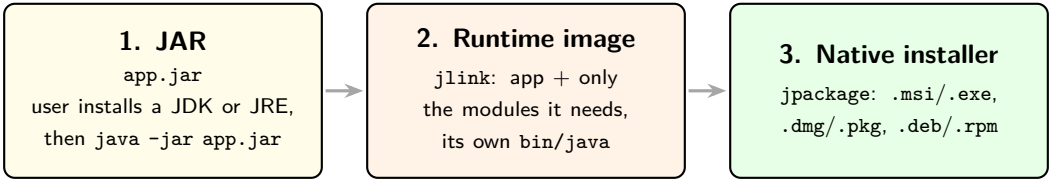
Modules (Java 9+)

```
// module-info.java
module edu.yeditepe.app {
    requires edu.yeditepe.util;
    requires java.sql;
}
```



- A **module** is a named group of packages with a `module-info.java`
- It declares what it **requires** and which packages it **exports**; others stay hidden
- The JDK itself is split into about 70 modules; `java.base` is always there
- Modules go on the **module path** (`-p`), run with `java -p mods -m edu.yeditepe.app`
- Most applications still use the class path; modules matter for libraries and for `jlink`

Three ways to deliver a Java application



- **JAR**: smallest, one file for every OS; needs Java on the user's computer
- **Runtime image**: no Java installation needed; one image per OS (built on that OS)
- **Installer**: feels like any other app (Start menu, Applications folder); `jpackage` must run on the target OS

Summary

Big ideas

Idea	In one sentence
Bytecode	`javac` compiles once to bytecode; any OS with a JVM runs it.
JDK	The JDK = runtime + tools; install a JDK (an LTS: 21 or 25) to develop.
PATH	The shell finds `javac` through the folders listed in `PATH`.
Package	A package is a folder tree; `java` runs a class by its full name.
Class path	`-cp` lists folders and JARs searched for classes (`;` on Windows, `:` on macOS).
JAR	A ZIP of classes and resources; `Main-Class` makes it runnable with `java -jar`.
Module	Named, encapsulated groups of packages; enable `jlink`.
Delivery	JAR, `jlink` runtime image, or `jpackage` installer.

Check yourself

1. **Q.** Which do you need to compile Java programs: a JRE or a JDK?
2. **Q.** `javac -version` works, but in a new folder `java Hello` says "Could not find or load main class". Why?
3. **Q.** Where does `javac -d classes` put `package a.b; class C`?
4. **Q.** Write the class path "folder `out` and `lib/x.jar`" for Windows and for macOS.
5. **Q.** What makes `java -jar app.jar` know which class to start?
6. **Q.** Your users have no Java installed. Which delivery options remain?

Check yourself

1. **Q.** Which do you need to compile Java programs: a JRE or a JDK? A JDK: the JRE has no `javac`.
2. **Q.** `javac -version` works, but in a new folder `java Hello` says "Could not find or load main class". Why?
3. **Q.** Where does `javac -d classes` put `package a.b; class C`?
4. **Q.** Write the class path "folder `out` and `lib/x.jar`" for Windows and for macOS.
5. **Q.** What makes `java -jar app.jar` know which class to start?
6. **Q.** Your users have no Java installed. Which delivery options remain?

Check yourself

1. **Q.** Which do you need to compile Java programs: a JRE or a JDK? A JDK: the JRE has no `javac`.
2. **Q.** `javac -version` works, but in a new folder `java Hello` says "Could not find or load main class". Why? There is no `Hello.class` on the class path (here: the current folder). Compile first or `cd`.
3. **Q.** Where does `javac -d classes` put `package a.b; class C`?
4. **Q.** Write the class path "folder `out` and `lib/x.jar`" for Windows and for macOS.
5. **Q.** What makes `java -jar app.jar` know which class to start?
6. **Q.** Your users have no Java installed. Which delivery options remain?

Check yourself

1. **Q.** Which do you need to compile Java programs: a JRE or a JDK? A JDK: the JRE has no `javac`.
2. **Q.** `javac -version` works, but in a new folder `java Hello` says "Could not find or load main class". Why? There is no `Hello.class` on the class path (here: the current folder). Compile first or `cd`.
3. **Q.** Where does `javac -d classes` put `package a.b; class C`? `classes/a/b/C.class`; run it with `java -cp classes a.b.C`.
4. **Q.** Write the class path "folder `out` and `lib/x.jar`" for Windows and for macOS.
5. **Q.** What makes `java -jar app.jar` know which class to start?
6. **Q.** Your users have no Java installed. Which delivery options remain?

Check yourself

1. **Q.** Which do you need to compile Java programs: a JRE or a JDK? A JDK: the JRE has no `javac`.
2. **Q.** `javac -version` works, but in a new folder `java Hello` says "Could not find or load main class". Why? There is no `Hello.class` on the class path (here: the current folder). Compile first or `cd`.
3. **Q.** Where does `javac -d classes` put `package a.b; class C`? `classes/a/b/C.class`; run it with `java -cp classes a.b.C`.
4. **Q.** Write the class path "folder `out` and `lib/x.jar`" for Windows and for macOS. `out;lib\x.jar` and `out:lib/x.jar`.
5. **Q.** What makes `java -jar app.jar` know which class to start?
6. **Q.** Your users have no Java installed. Which delivery options remain?

Check yourself

1. **Q.** Which do you need to compile Java programs: a JRE or a JDK? A JDK: the JRE has no `javac`.
2. **Q.** `javac -version` works, but in a new folder `java Hello` says "Could not find or load main class". Why? There is no `Hello.class` on the class path (here: the current folder). Compile first or `cd`.
3. **Q.** Where does `javac -d classes` put `package a.b; class C`? `classes/a/b/C.class`; run it with `java -cp classes a.b.C`.
4. **Q.** Write the class path "folder `out` and `lib/x.jar`" for Windows and for macOS. `out;lib\x.jar` and `out:lib/x.jar`.
5. **Q.** What makes `java -jar app.jar` know which class to start? The `Main-Class` line in `META-INF/MANIFEST.MF`.
6. **Q.** Your users have no Java installed. Which delivery options remain?

Check yourself

1. **Q.** Which do you need to compile Java programs: a JRE or a JDK? A JDK: the JRE has no `javac`.
2. **Q.** `javac -version` works, but in a new folder `java Hello` says "Could not find or load main class". Why? There is no `Hello.class` on the class path (here: the current folder). Compile first or `cd`.
3. **Q.** Where does `javac -d classes` put `package a.b; class C`? `classes/a/b/C.class`; run it with `java -cp classes a.b.C`.
4. **Q.** Write the class path "folder `out` and `lib/x.jar`" for Windows and for macOS. `out;lib\x.jar` and `out:lib/x.jar`.
5. **Q.** What makes `java -jar app.jar` know which class to start? The `Main-Class` line in `META-INF/MANIFEST.MF`.
6. **Q.** Your users have no Java installed. Which delivery options remain? A `jlink` runtime image or a `jpackage` installer.

Questions?